IOE413 — STREAMING DATA ANALYTICS

Mini Project — Guidelines

Design, build, and evaluate an end-to-end streaming analytics pipeline that extends 2–3 published base papers on your assigned topic, using tools from this course — with real data actually flowing between them.

1. Requirements

- **Team size.** 4 students per team.
- **Base papers.** Each team selects 2–3 peer-reviewed papers relevant to its assigned topic. The project must reproduce (at least a simplified version of) one paper's core method as a baseline, then build a clearly-scoped enhancement on top of it.
- **Problem scope.** Choose a streaming dataset/source and question substantial enough that a single tool, or a batch-only approach, would genuinely struggle with it — because of data velocity, the need for approximate/online computation, or a technique (ML, quantum, generative, causal, agentic) the other tools don't provide.
- **Streaming source.** Use a real or realistically-simulated stream (Kafka/MQTT topic replaying a public dataset — e.g., NAB, SWaT, a Kaggle IoT/sensor dataset, or a synthetic generator — is acceptable; a static CSV read once is not.

2. Assigned Topics

Your team will be assigned one of the following.

- 1. Adaptive Probabilistic Data Structures for Resource-Constrained Edge Streaming Analytics
- 2. Approximate Streaming Analytics on Resource-Constrained Edge Devices
- 3. Real-Time Digital Twin for Streaming-Based Anomaly Detection
- 4. Quantum vs Classical Approaches for Streaming Data Analytics: An Experimental Study
- 5. Diffusion Models for Real-Time Imputation of Missing IoT Sensor Streams
- 6. An Agentic AI Framework for Autonomous Streaming Data Analytics
- 7. Retrieval-Augmented Generation for Explainable Real-Time Streaming Analytics
- 8. SketchRAG: Approximate Streaming Summaries as Retrieval Context for Agentic AI
- 9. Federated Sketches for Privacy-Preserving Streaming Analytics
- 10. Continual Learning for Concept-Drifting Data Streams
- 11. Causal Inference for Real-Time Streaming Event Analytics
- 12. Detection of concept drift in industrial data streams
- 13. Concept Drift Detection in Social Media Streams
- 14. Streaming graph mining

3. Suggested Tool Combinations

Some ideas for tools that can be used in the project are given for your reference.

Track & core idea	Suggested tool pipeline
Track A — Probabilistic Sketches on the Edge	MQTT / Kafka (ingest) → Flink or a lightweight engine (Faust / Bytewax) (windowed processing) → DataSketches / Algebird — Bloom filter, Count-Min Sketch, HyperLogLog (approximate structures) → Redis or InfluxDB + Grafana (state store & live dashboard)
Track B — Streaming ML for Anomaly & Drift	Kafka (ingest) → Spark Structured Streaming or Flink (feature windows) → River (online learning, ADWIN/DDM drift detectors) or PyTorch (autoencoder) (model) → InfluxDB/Elasticsearch + Grafana/Kibana (live dashboard / digital-twin view)
Track C — Generative & Quantum Methods	Kafka / MQTT (ingest) → Spark Structured Streaming (classical baseline features) → Qiskit or PennyLane (quantum circuits) or PyTorch (diffusion model) (core technique) → InfluxDB + Grafana (results dashboard)
Track D — Agentic AI & RAG for Streams	Kafka (ingest) → Spark or Flink Streaming (windowed summarization) → FAISS / Chroma / Milvus (vector store) → LangChain or LangGraph with an LLM API (agentic / RAG layer)
Track E — Causal & Explanatory Streaming Analytics	Kafka (ingest) → Spark Structured Streaming (event windows) → DoWhy / CausalImpact / EconML (causal estimation) → Grafana or Kibana (dashboard of causal effects over time)

4. Base Papers & Enhancement

- **Selection.** 2–3 papers, at least one of which proposes a method your team will reproduce as a baseline
- **Enhancement.** Clearly state, in one or two sentences, what your team is changing relative to the base paper(s) — e.g., a new sketch structure, a lighter-weight model for edge deployment, an added drift-detection layer, a causal/explainability layer, a different retrieval strategy, etc.
- **Comparison.** Final results must include a quantitative comparison between the reproduced baseline and your enhancement, on the metrics that matter for the topic (accuracy/error, latency, memory footprint, throughput, privacy leakage, drift-recovery time, etc., as appropriate).
- Submit the papers for instructor approval before proceeding.

5. Deliverables

- **Proposal.** Problem statement, 2–3 base papers with a one-paragraph summary each, dataset/stream source, metrics, chosen tool combination (3+), and the expected enhancement. The proposal should include for each base paper:
    a. Research question
    b. Data used
    c. Metrics used
    d. Key findings
    e. Model (if any)
- **Code + README.** The full pipeline, with setup and run instructions and a list of tools/versions used.
- **Final report.** 6–8 pages, IEEE/ACM two-column conference format (see Section 6) — architecture, base-paper summary and gap, enhancement, experimental setup, results, inferences. The PDF and the editable files shall both be submitted.
- **Demo.** Live walkthrough or a video no longer than 10 minutes, showing the full pipeline actually running end to end on streaming data.

6. Final Report Format — IEEE/ACM Conference Style

The final report must be formatted as a conference paper: two-column layout, 6–8 pages including references and figures, using the standard IEEE or ACM conference template.

Required sections:

- Title, author names, affiliation, abstract (150–250 words)
- I. Introduction — motivation, problem statement, contributions
- II. Related Work — summary of the 2–3 base papers and the specific gap your enhancement addresses
- III. System Architecture — the full pipeline, with an architecture diagram showing every tool and the data flow between them
- IV. Proposed Enhancement — what is new relative to the base paper(s), and why
- V. Implementation — dataset/stream, tool versions, key configuration choices
- VI. Experimental Setup & Results — metrics, baseline vs. enhancement comparison, tables/plots
- VII. Discussion / Inferences — what the results mean, limitations, threats to validity
- VIII. Conclusion & Future Work
- References — IEEE or ACM citation style, matching the chosen template

Reports outside the 6–8 page window, or not in two-column IEEE/ACM format, will be returned for reformatting before grading.

Plagiarism will result in zero marks.

7. Evaluation — Out of 50

This mini project is worth 20% of the final course grade.

Component	Points	Rubrics
Proposal & Literature Review	**5**	Clear problem statement; 2–3 base papers correctly identified and summarized; feasibility and tool-choice justification.
Implementation & Tool Integration	**15**	Correctness of the pipeline; genuine use of streaming data tools; real data flow between stages, not isolated scripts.
Enhancement / Novelty	**8**	The enhancement over the base paper(s) is clearly articulated, non-trivial, and actually implemented — not just proposed.
Results & Evaluation	**7**	Sound experimental design; meaningful metrics; comparison against the base paper's reported results or a reproduced baseline; honest reporting of negative results.
Report Quality & IEEE/ACM Formatting	**7**	6–8 pages, two-column IEEE/ACM conference format, complete sections, architecture diagram, reproducible setup instructions, proper citations.
Demo & Presentation	**3**	Live walkthrough or ≤10-minute video showing the pipeline actually running end to end; clarity of delivery.
Q&A / Defense of Design Choices	**5**	Ability to explain and defend tool choices, enhancement, and results under questioning.
Total	**50**	

Publication or open source contributions will fetch additional credits.

9. Suggested Timeline

Suggested week(s)	Activities
Sep 09-10	Team formation, topic confirmation
Sep 10-Sep 16	Literature search. Submit Proposal: problem statement, 2–3 base papers, dataset/stream source, chosen tool pipeline, expected enhancement.
Sep 17-Sep 23	Set up ingestion + storage layer; reproduce the base paper's core method as a working baseline.
Sep 24-Oct 7	Design and implement the enhancement; build the analytical/ML/agentic component on top of the baseline.
Oct 8-Oct 14	Run full experiments, collect metrics, compare against baseline, write inferences. Write the IEEE/ACM-format final report; finalize the README.
Oct 15-Oct 21	Record/rehearse the demo; final submission and in-class presentation & Q&A.